CRISIS ROOM

PROJECT BRIEF

Crisis Room

Automated incident response — detect, diagnose, decide, resolve.

WHAT IT IS	A platform that runs the full incident-response workflow automatically – from detecting a failing service to applying the fix and confirming recovery.
WHO IT'S FOR	Engineering and operations teams who carry a pager: anyone responsible for keeping a production service up.
STATUS	Working implementation, verified end to end. Not a mockup.
TEAM	Gol_Gappe – Riddhika Sachdeva & Hemank, MAIT, Delhi.

Build with Bharat 2.0 · National Level Hackathon
NIT Delhi · organized by CodeVerse

CONTENTS

01

Summary

Crisis Room handles a production incident the way a well-drilled on-call team would — except four specialized agents share the work, move in seconds, and show every step as it happens.

When a monitored service starts failing, Crisis Room opens an incident on its own and works it end to end: it classifies how severe the problem is, works out the most likely cause, chooses a single remediation and explains why, applies that fix, watches the service recover, and drafts the updates that customers, engineers, and leadership each need. Every stage is streamed live to a dashboard, and every decision is recorded and inspectable.

It plugs into the monitoring tools a team already runs, and its execution layer is built so the same system that fixes a sandboxed test service today can drive real infrastructure tomorrow without a rewrite.

02

The problem

When a service breaks at 2:47 in the morning, one on-call engineer has to do four jobs at once, under a clock, while revenue and users drain away:

- **Assess** — how bad is this, and who needs to be pulled in?
- **Investigate** — what actually changed? A deploy, a dependency, capacity, the network?
- **Decide** — roll back, restart, scale, fail over — and be right the first time.
- **Communicate** — keep customers, the engineering channel, and leadership informed, each in the register they expect.

The context needed to do this is scattered across dashboards, logs, and chat threads. The steps are done in sequence when they could be done in parallel. And the whole response depends on the judgment of whoever happens to be holding the pager that night. Minutes of delay are measured in money and trust.

03

What Crisis Room does

Crisis Room automates the entire response loop. You tell it which services to watch. From then on:

- It **watches** those services and opens an incident the moment one genuinely degrades — not on a single blip, but on sustained failure.
- It **runs the response** through four specialized agents that hand off to each other through a strict shared format.
- It **executes** the chosen fix through a pluggable action layer and measures the real recovery, tick by tick.
- It **reports** the outcome by email and Slack, and keeps a full, replayable timeline of every incident.
- It **shows its work** — a live dashboard renders each agent's output and its decision log as it is produced.

Nothing here waits for a person to click a button. The scripted walkthroughs in the demo exist only so a presentation tells the same story every time; the real product reacts on its own.

04

The four-agent pipeline

An incident flows through four agents in order. The Communicator runs twice — once immediately, so stakeholders are never left in silence, and again at the end with the resolution.

AGENTS — JOB, INPUT, OUTPUT

AGENT	JOB	TAKES IN	PRODUCES
Triage	Classify severity fast and flag what to look at	The raw alert or monitoring signal	A severity level (`SEV1–SEV4`), affected services, and leads for the Investigator
Investigator	Find the most likely root cause	Triage's output and the signal detail	A specific hypothesis, the evidence for it, and the causes it ruled out and why
Commander	Choose one remediation and justify it	Triage and Investigator output	One action — `rollback` / `restart` / `scale` / `failover` / `escalate` / `monitor` — with rationale, expected impact, and a rollback plan
Communicator	Draft stakeholder updates	Current incident state	Three messages — customer status page, engineering channel, leadership — each in its own register, plus a next-update time

Every agent's output is validated against one shared schema before it is allowed to reach the next agent or the dashboard. That single typed contract is what keeps a multi-stage pipeline predictable instead of turning into a chain of loosely connected guesses.

05

The incident lifecycle

Every incident, whatever its source, moves through the same seven stages.

01 **Detect**

A watched service fails its checks for long enough to count as a real problem, or an external monitoring tool fires a webhook. An incident is opened and tagged to the service and its owner.

02 **Triage**

Severity is assigned from the measured impact — error rate, latency, blast radius — and the incident is routed with initial leads.

03 **Notify (holding)**

A first “we’re on it” message is drafted straight away, so no audience sits in silence while the cause is still being found.

04 **Investigate**

The failure’s shape is matched against known incident signatures across infrastructure, network, and dependency categories, and a root-cause hypothesis is committed to with its evidence.

05 **Decide**

One remediation is selected and fully explained — why this action, why not the alternatives, and how to reverse it.

06 **Execute & confirm**

The action is applied through the execution layer and the service’s recovery is measured over real time until the error rate comes back to healthy — or clearly does not.

07 **Resolve & report**

The incident is closed with an honest final status, a resolution report is sent by email and Slack, and the full timeline is stored for the postmortem.

06

Detection — two ways in

Built-in synthetic monitoring

Add the URL of a service you want watched. Crisis Room polls it on an interval, keeps a rolling window of the actual status codes and response times it observed, and opens an incident only when that window shows genuine, sustained degradation — a run of failures, a sustained error rate, or a latency ceiling breached — never on a single slow response. When the service recovers, the monitor re-arms itself. If the failure changes character mid-incident, that is treated as a new problem.

Webhook ingestion from existing tools

Point a monitoring tool a team already runs — Datadog, PagerDuty, Prometheus Alertmanager, or any in-house alerting — at a single endpoint. A small per-source adapter translates that tool's payload into Crisis Room's one incident format, and the full pipeline runs with no one touching anything. Supporting a new tool means writing one adapter function; nothing else in the system changes.

> **One always-on connection.** A dashboard keeps a single live channel open. Any incident, from any source, appears on it automatically and updates until it resolves — nobody needs to know an incident's ID in advance.

07

Diagnosis and decision

Investigator — root cause

A black-box monitor cannot read a team's deploy history or internal logs, but it can see the *shape* of a failure: the mix of status codes, how far latency has moved from baseline, whether requests are erroring or timing out, and when the onset was. The Investigator matches that shape against a library of known incident signatures — a code regression from a recent change, capacity exhaustion, an unreachable upstream dependency, a network fault — and commits to the best fit, stating explicitly which other causes it ruled out and on what grounds. That breadth is what makes the diagnosis trustworthy rather than a single guess.

Commander — the call

The Commander selects exactly one action from a fixed set — `rollback, restart, scale, failover, escalate, monitor` — and records the rationale in plain language, the expected impact, a confidence level, and a rollback plan. It never returns a bare action with no reasoning; the explanation is the point. Its selection criteria come from a scoring rubric carried over from the team's earlier incident-response work:

WHAT "GOOD INCIDENT COMMAND" IS SCORED ON

CRITERION	WEIGHT	MEASURES
Resolution correctness	35%	Right root cause *and* right remediation
Time efficiency	20%	A faster correct resolution scores higher

CRITERION	WEIGHT	MEASURES
Communication quality	20%	Stakeholders updated at the right intervals
Routing accuracy	15%	The right questions sent to the right analysis path
Postmortem quality	10%	Cause, impact, and action items stated correctly

08

Real remediation

Crisis Room does not stop at a recommendation. The Commander's action is handed to an execution layer that carries it out, and the target's recovery is then measured — not projected — over real wall-clock time.

Against a service Crisis Room controls

The build ships with a live target service that has real, injectable faults and a real control interface. When the Commander picks `rollback` and the executor calls the control interface, that service genuinely stops failing and its error rate genuinely comes down on the next polls. If the Commander picks the wrong action, it genuinely does not — the recovery chart stays flat at the real, unchanged error rate. The full loop — decide, execute, confirm — is proven, not simulated.

Against an external service

For a service Crisis Room has no control channel to, it is honest about the boundary: the diagnosis and the recommended action are delivered in full, the recovery view is clearly labelled a projection rather than a measurement, and a person or a connected automation carries out the fix.

Against real infrastructure

The agents, the orchestrator, and the dashboard only ever talk to one execution interface. Swapping the sandbox target for a real Kubernetes cluster, cloud API, or deploy pipeline is a single new implementation of that interface — nothing upstream changes. A recommend-only mode, where the decision is logged but nothing is touched, is a one-setting switch.

09

Honest outcomes

An incident is only marked resolved when the service is actually measured healthy again — not just because the pipeline finished running. Every incident ends in one of three states:

RESOLVED	The remediation was applied and the service's measured error rate came back to healthy.
MITIGATION FAILED	The remediation was applied but the service is still degraded — the wrong action, or a cause outside Crisis Room's control. Flagged for a person to take over.
AWAITING EXECUTION	Diagnosed and decided, but there is no control channel to carry the fix out. The recovery shown is a projection.

This matters because a system that always reports success is worse than useless during a real incident. The dashboard surfaces the count of incidents that need attention, not just the count that closed.

10

Communication

Silence during a live incident is itself a failure. The Communicator drafts three messages for every incident, each written for its audience:

- **Customer status page** — plain language, apologetic but not alarmist, no internal jargon.
- **Engineering channel** — technical and terse: root cause, the action being taken, why.
- **Leadership** — framed on business impact and expected recovery time, not implementation detail.

A holding message goes out right after triage; the full report goes out on resolution. Delivery is by email to the account owner and, optionally, to a Slack channel via an incoming webhook the user configures in settings. Each incident record keeps a log of exactly what was sent and where.

11

Two surfaces

The live console

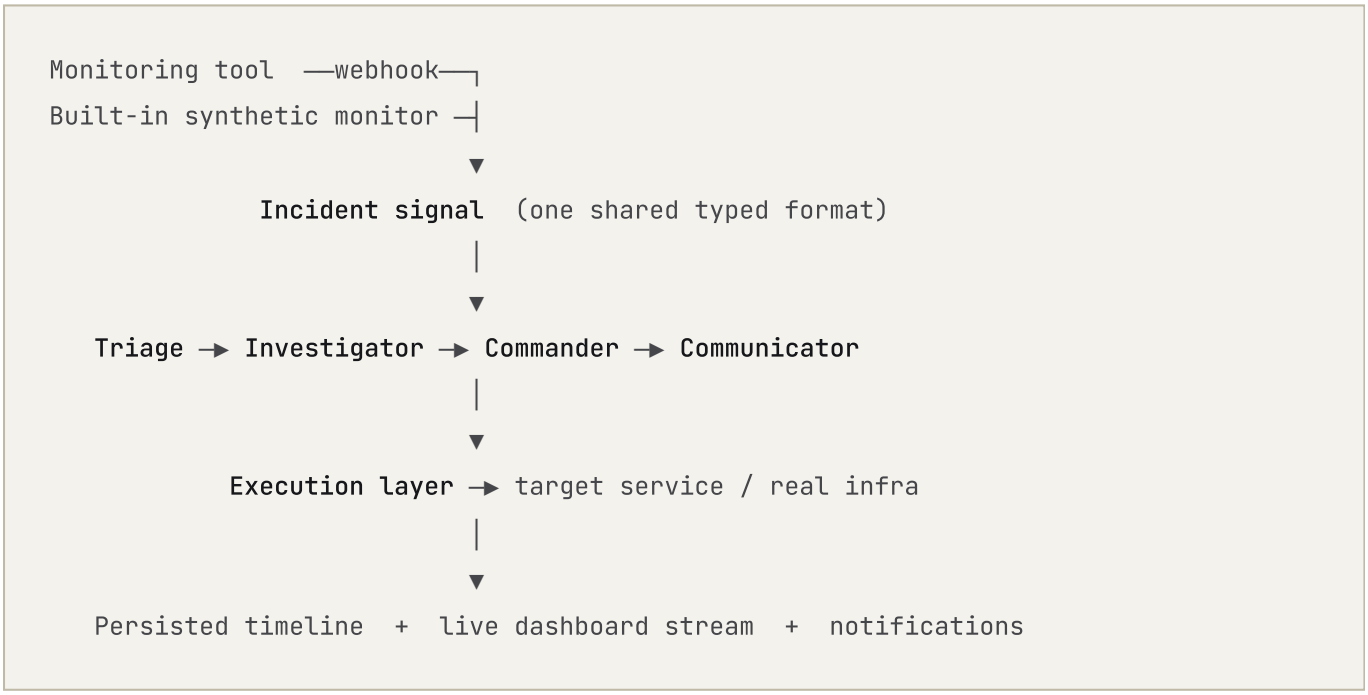
Open, no login, always listening. It is what an operator leaves open all day: an incident feed that populates itself, and for each incident a pipeline rail, a panel per agent showing its output, a running decision log, and a live recovery chart. Three scripted scenarios — a capacity outage, a bad deploy, a network partition — give a repeatable walkthrough that reaches a different correct decision each time from different evidence.

The accounts area

Sign in with an email and password. Register the services you want watched and set each one's thresholds and poll interval. Browse your incident history, open any incident to replay its full timeline stage by stage, and configure where resolution reports are delivered. Each user sees only their own services and incidents.

12

System architecture



COMPONENTS

COMPONENT	RESPONSIBILITY

COMPONENT	RESPONSIBILITY
Orchestrator	Runs the four agents in order and emits each result the instant it is produced
Agents	Triage, Investigator, Commander, Communicator — each a self-contained unit with one job and a typed output
Adapters	Normalize Datadog / PagerDuty / Prometheus / generic webhook payloads into the shared incident format
Synthetic monitor	One background task per watched service; polls, keeps a rolling window, decides when a real incident exists
Execution layer	Applies the chosen action and streams measured recovery; one interface, swappable backend
Target service	A controlled service with injectable faults and a control interface, for proving the loop end to end
API layer	REST endpoints plus WebSocket streams for the live feed and per-incident replay
Persistence	Accounts, watched services, incidents, and the ordered event log for each incident
Dashboard	Renders the live stream: pipeline rail, agent panels, decision log, recovery chart

13

Data model

RECORD	HOLDS
`users`	Email, hashed password, optional Slack webhook, created date
`monitored services`	Owner, URL, display name, thresholds, poll interval, active flag
`incidents`	Owner, service, source, title, severity, status, the triggering signal, the resolution summary, timestamps
`incident events`	Every agent output and lifecycle event for an incident, stored in order — the replayable timeline

Because every step is persisted as it happens, an incident detail view is a faithful replay of what occurred, in sequence, not a summary reconstructed after the fact.

Design principles

One typed contract between every stage

Nothing crosses from one agent to the next as free text. Every output is validated against a shared schema first. This is the single biggest reason a multi-stage pipeline stays reliable.

A deterministic core

The classification and decision logic runs fully offline, with the same typed outputs, against a library of incident signatures and thresholds. A weak venue network cannot take down a demo, and a production deployment does not hinge on an external service being reachable.

One execution interface

The agents, orchestrator, and dashboard know nothing about how a fix is carried out. Moving from a sandbox to real infrastructure — or inserting a human-approval gate — is a change in exactly one place.

The demo surface is frozen

The public console and its scripted scenarios never write to the database, never send real email, and never touch real infrastructure. The product features are built strictly alongside it.

Security

Accounts use email and password with hashed storage. Sessions are signed tokens held in http-only cookies. Every read is scoped to the requesting user.

15

Technology

Backend

Python 3	language
FastAPI	REST + WebSocket
Uvicorn	ASGI server
SQLAlchemy 2	ORM
SQLite	embedded database
Pydantic 2	typed contracts
pydantic-settings	configuration
httpx	async HTTP client
PyJWT	session tokens
passlib + bcrypt	password hashing
Resend	email delivery

Frontend

Next.js 14	App Router
React 18	UI
CSS variables + styled-jsx	scoped styling
lucide	icon set
Custom SVG	brand mark

Interfaces & patterns

WebSocket streaming	live feed + replay
Webhook ingestion	per-source adapters
Async background tasks	one per watched service
JWT in http-only cookie	sessions
Single typed schema	every pipeline stage

Tooling & deployment

Docker	backend container
Playwright	headless visual QA
python-dotenv	environment config
Next build pipeline	static + server

16

What is built today

Everything described in this brief is implemented and verified working end to end:

- Both detection paths — built-in synthetic monitoring and webhook ingestion for Datadog, PagerDuty, Prometheus, and generic sources.
- The complete four-agent pipeline with the shared typed contract and the deterministic offline core.
- Real execution against the controlled target service, proven for every fault type — the correct action fixes it, a wrong action measurably does not.
- The honest three-state outcome logic, driven by measured recovery.
- Email delivery of resolution reports, plus the optional Slack channel.
- The always-on live console with pipeline rail, agent panels, decision log, and live recovery chart.
- The accounts area — registration, per-service monitoring configuration, incident history, full timeline replay, notification settings.
- Persistence of every incident and every step within it.

17

Roadmap

- **Production execution backends** — ready-made implementations of the execution interface for Kubernetes, major cloud providers, and common deploy pipelines.
- **Human-approval gate** — an optional confirmation step before any action touches real infrastructure, which is what most organizations require.
- **More monitoring integrations** — additional source adapters; each is a single function.
- **Multi-instance deployment** — move the background monitoring and pipeline work behind a shared queue so the platform scales past one process.
- **Richer postmortems** — auto-drafted incident write-ups with timeline, impact, and action items from the stored event log.

Appendix

Running it locally

```
Backend   pip install -r requirements.txt
          uvicorn server.main:app --port 8000

Frontend  cd frontend && npm install && npm run dev
          open http://localhost:3000
```

Key endpoints

ENDPOINT	PURPOSE
`POST /api/webhooks/{source}`	Ingest an alert from an external monitoring tool
`WS /ws/live`	Always-on stream of every incident as it happens
`POST /api/auth/signup`	Create an account
`POST /api/sites`	Register a service to monitor
`GET /api/incidents`	The signed-in user's incident history

Suggested eight-slide breakdown

SLIDE	DRAW FROM
1 Problem	Section 02
2 Solution	Sections 01, 03
3 How it works – the pipeline	Sections 04, 05
4 Detection & integration	Section 06
5 Real remediation & honest outcomes	Sections 08, 09
6 Architecture	Section 12

SLIDE	DRAW FROM
7 Technology & what's built	Sections 15, 16
8 Roadmap & impact	Section 17